

Atividade Final - Paradigmas de Linguagens Programação

Informações do Autor

- **Nome:** Roussian Gaioso
 - **Data de atualização:** 15/05/2026
-

Regras de Entrega

- Grupos no máximo de 6 integrantes;
 - Trabalhos iguais de grupos diferentes, nota zero;
 - Qualquer tentativa de fraude implicará em nota 0,0 (zero) na atividade, sem prejuízo de outras sanções;
 - O grupo é responsável pela submissão, assim arquivos corrompidos ou ausência de arquivos impactará a nota ou a zerará;
 - Os nomes dos integrantes devem estar no arquivo;
 - Caso a submissão seja reaberta, os trabalhos submetidos após a data prevista terão suas notas reduzidas pela metade;
 - Esse trabalho contará como AED e um percentual da **Nota de Atividades**, a ser definido no final do bimestre.
-

Descrição

Cada grupo deverá implementar um pequeno sistema de análise de desempenho acadêmico em diferentes linguagens ou paradigmas de programação.

O sistema deverá receber dados de alunos, notas e frequência, processar essas informações e gerar um relatório final com a situação de cada aluno.

O problema deverá ser resolvido em mais de uma abordagem, para permitir a comparação entre os paradigmas.

Dados de entrada

Cada aluno deverá possuir, no mínimo:

nome
matrícula
nota 1
nota 2
nota 3
frequência

Exemplo de dados:

Ana;2024001;8.0;7.5;9.0;85
Bruno;2024002;5.0;6.0;4.5;80
Carlos;2024003;7.0;6.5;8.0;60
Daniela;2024004;9.0;9.5;10.0;95

Regra de aprovação

A média final deverá ser calculada da seguinte forma:

$$\text{media} = (\text{nota1} + \text{nota2} + \text{nota3}) / 3$$

O aluno será considerado aprovado se:

$$\text{media} \geq 6.0$$
$$\text{frequencia} \geq 75$$

Caso contrário, será considerado reprovado.

Parte 1 — Implementação imperativa

Cada grupo deverá implementar uma versão imperativa do programa.

Linguagem sugerida:

C

A implementação deverá conter:

variáveis;
atribuições;
expressões aritméticas;
comandos condicionais;
laços de repetição;
funções;
leitura dos dados;
cálculo da média;
classificação dos alunos;
impressão do relatório final.

Parte 2 — Implementação orientada a objetos

Cada grupo deverá implementar uma versão orientada a objetos do mesmo problema.

Linguagens sugeridas:

Java
Python
C++

A implementação deverá conter, no mínimo:

classe Aluno;
atributos para nome, matrícula, notas e frequência;
método para calcular média;
método para verificar aprovação;
classe Turma ou Relatorio;
encapsulamento dos dados;
uso de objetos;
chamada de métodos.

Parte 3 — Implementação Funcional

Cada grupo deverá implementar uma versão funcional ou predominantemente funcional do mesmo problema.

Linguagens sugeridas:

Python
JavaScript
Haskell
Elixir

A implementação deverá conter:

- funções puras sempre que possível;
- uso de listas;
- uso de map, filter ou reduce;
- evitar alteração direta de variáveis globais;
- separação entre entrada, processamento e saída;
- função para calcular média;
- função para verificar aprovação;
- função para gerar a lista de resultados.

Parte 4 — Implementação Lógica - EXTRA

Cada grupo deverá implementar uma versão declarativa do problema.

Linguagens sugeridas:

Prolog
Datalog

A versão declarativa poderá representar os alunos como fatos.

Exemplo em Prolog:

```
aluno(ana, 2024001, 8.0, 7.5, 9.0, 85).  
aluno(bruno, 2024002, 5.0, 6.0, 4.5, 80).
```

Depois, deverá criar regras para:

- calcular média;
- verificar frequência;
- determinar aprovação;
- consultar alunos aprovados;
- consultar alunos reprovados.

Exemplo conceitual:

```
aprovado(Nome) :-  
    aluno(Nome, _, N1, N2, N3, Frequencia),  
    Media is (N1 + N2 + N3) / 3,  
    Media >= 6.0,  
    Frequencia >= 75.
```

A versão declarativa deverá mostrar que o foco está em declarar fatos e regras e não em escrever uma sequência detalhada de comandos.

Parte 5 — Relatório

Cada grupo deverá entregar um relatório técnico comparando as implementações.

O relatório deverá conter as seguintes seções.

1. Descrição do problema

Apresentar o problema resolvido, os dados de entrada, as regras de aprovação e o resultado esperado.

2. Linguagens escolhidas

Informar quais linguagens foram utilizadas e justificar a escolha.

Exemplo:

```
C para a versão imperativa.  
Java para a versão orientada a objetos.  
Python para a versão funcional.  
Prolog para a versão lógica.
```

3. Comparação de legibilidade

Comparar qual versão ficou mais fácil de ler.

A análise deverá considerar:

```
clareza dos nomes;  
organização do código;  
quantidade de detalhes sintáticos;  
facilidade para entender o fluxo do programa;  
facilidade para localizar onde cada regra foi implementada.
```

4. Comparação de facilidade de escrita

Comparar qual versão foi mais fácil de implementar.

A análise deverá considerar:

- quantidade de código;
- dificuldade da sintaxe;
- facilidade para manipular listas ou arquivos;
- necessidade de criar estruturas auxiliares;
- uso de bibliotecas.

5. Comparação de confiabilidade

Comparar qual versão parece mais segura contra erros.

A análise deverá considerar:

- verificação de tipos;
- risco de erro em tempo de execução;
- tratamento de entrada inválida;
- risco de acesso indevido à memória;
- facilidade para testar cada parte do programa.

6. Comparação sobre nomes, escopo e tempo de vida

O grupo deverá escolher exemplos do próprio código e explicar:

- quais nomes foram declarados;
- onde esses nomes são visíveis;
- qual é o escopo de variáveis, funções, métodos ou predicados;
- quando os dados são criados;
- quando deixam de existir.

7. Comparação sobre tipos de dados

O grupo deverá comparar como cada linguagem tratou os dados.

A análise deverá considerar:

tipagem estática ou dinâmica;
tipos primitivos;
strings;
listas, vetores ou estruturas;
objetos;
conversões de tipo;
erros de tipo.

8. Comparação entre os paradigmas

O grupo deverá comparar diretamente os paradigmas.

A comparação deverá responder:

Como a versão imperativa organiza a solução?
Como a versão orientada a objetos organiza a solução?
Como a versão funcional organiza a solução?
Como a versão declarativa organiza a solução?
Qual versão ficou mais próxima da forma humana de descrever o problema?
Qual versão ficou mais próxima da forma como a máquina executa os passos?
Qual versão parece mais adequada para esse tipo de problema?

9. Conclusão

O grupo deverá apresentar uma conclusão crítica.

A conclusão não deve apenas dizer qual linguagem é “melhor”.

Deverá explicar em quais situações cada paradigma parece mais adequado.

Entrega

Cada grupo deverá entregar:

código da versão imperativa;
código da versão orientada a objetos;
código da versão funcional;
código da versão declarativa ou lógica;
arquivo de entrada usado nos testes;
prints ou saídas das execuções;
relatório técnico comparativo.